

누구에게 어떤 프로모션을 제공해야 할까?

예산 제약 하의 정책학습(Policy Learning) 기반 프로모션 타겟팅

데이터 분석 포트폴리오

조해창 GitHub: github.com/Funbucket LinkedIn: [hae-chang-cho](#)

관찰 데이터 · 다중 처치 · Doubly Robust(AIPW) · econml DRLearner/DRPolicyTree/DRPolicyForest

요약 (Executive Summary)

문제. 한 B2B 소프트웨어 기업이 고객사에게 기술지원·할인 프로모션을 제공한다. 예산은 제한되어 있고, 프로모션 효과는 고객사마다 다르다. 이 조건에서 단일 정책은 최선이 아닐 수 있다.

이 분석의 목적은 단일 정책 대신 고객사별로 순수익을 최대화하는 처치 배정 규칙을 도출하고, 예산 수준에 따라 어떤 정책이 더 효율적인지 평가하는 것이다.

접근. 관찰 데이터에 정책학습(policy learning)을 적용해 고객사별 최적 처치 규칙을 학습하고, doubly robust(AIPW) 기준으로 정책 가치를 평가했다. 예산 제약 하의 효율은 비용 곡선(cost curve)으로 비교했다.

핵심 결과.

정책	고객사당 평균 순이익	처치 없음 대비
처치 없음 (all none)	\$7,249	—
최선의 단일 처치 (전원 기술지원)	\$10,527	+45%
고객사별 타겟팅 (DRLearner plug-in)	\$11,798	+63%

- 고객사별 타겟팅 정책은 가장 좋은 단일 처치 정책보다 약 12%, 처치 없음 대비 약 63% 높은 순이익을 냈다.
- 처치 배정을 좌우한 핵심 요인은 직원 수와 고객사 규모였다. 규모가 큰 고객사에게는 기술지원 + 할인을, 그 외 고객사에게는 주로 기술지원을 배정했다.
- 예산 수준에 따라 최적 정책이 달랐다. \$3.9M 이하에서는 전원 기술지원이, \$3.9M~\$6.5M에서는 학습 정책이, \$6.5M~\$9.3M에서는 기술지원+할인 정책 내 고효율 고객사 선별이 가장 효율적이었다.

핵심 제안. 모두에게 동일한 프로모션을 적용하기보다, 고객사 규모·직원 수에 기반한 타겟팅 규칙을 적용하길 제안한다. 성능이 최우선이면 plug-in/forest 정책을, 설명·운영 안정성이 중요하면 해석 가능한 depth=2 트리 정책을 권한다.

1. 비즈니스 문제

이 분석의 상황은 B2B 소프트웨어 고객사 대상 프로모션 의사결정이다. 회사는 고객사에게 기술지원과 할인을 각각 제공할 수 있고, 두 개입을 조합하면 처치 없음, 기술지원만, 할인만, 기술지원 + 할인 네 가지 선택지가 생긴다. 관측된 결과는 프로모션 이후 1년 동안의 소프트웨어 구매 매출이며, 의사결정의 목표는 매출이 아니라 비용까지 뺀 순이익이다.

인과추론의 많은 문제는 평균 처치 효과(ATE)에 집중한다. “프로모션을 진행할 것인가, 하지 않을 것인가?”라는 질문은 모든 고객사에게 동일한 프로모션을 제공하는 정책과 아무에게도 제공하지 않는 정책을 비교하는 문제다.

하지만 예산이 제한되어 있고 그 안에서 최대 효과를 내야 한다면 이야기가 달라진다. 프로모션 효과가 고객사마다 다르다면, 모두에게 동일하게 제공하는 전략이 최선이 아닐 수 있다.

- 마케팅 유무와 상관없이 어차피 구매하거나(*always-converters*), 반대로 아무리 혜택을 줘도 사지 않을 고객사(*lost causes*)에게 불필요한 예산을 낭비하게 된다.
- 마케팅에 부정적으로 반응하는 고객사(*sleeping dogs*) 때문에 오히려 손실이 발생하며,
- 정작 효과가 클 고객사(*high-responders*)를 우선순위에 두지 못해 잠재 매출을 놓친다.

따라서 이 분석이 답하려는 질문은 단순히 “프로모션이 평균적으로 효과가 있는가?”가 아니라 “어떤 고객사에게 어떤 프로모션을 배정해야 순이익이 가장 커질까?”이다. 가능한 처치 규칙들을 탐색해 전체 평균 순수익을 근사적으로 최대화하는 문제, 즉 “누구에게 무엇을 배정해야 하는가?”에 답하는 문제를 정책학습(policy learning)이라고 부른다.

이 분석은 다음 다섯 가지 질문으로 구성된다.

- Q1. 프로모션 효과는 정말 고객사마다 다른가? (이질성 진단)
- Q2. 관찰 데이터로 인과 효과를 신뢰할 수 있는가? (식별 가정)
- Q3. 누구에게 무엇을 배정해야 하는가? (정책학습)
- Q4. 학습한 정책이 정말 더 나은가? (정책 평가)
- Q5. 예산이 제한되면 어떤 정책이 유리한가? (예산 제약)

2. 데이터와 처치 정의

데이터는 Microsoft의 공개 데이터를 사용했다. 각 행은 한 고객사를 나타내며, 약 2,000곳의 고객사 정보로 구성된다. 고객사 특성은 글로벌 오피스 보유 여부, 대규모 소비 고객사 여부, 중소기업 여부,

상업용 사업 여부, IT 지출, 직원 수, PC 수, 연간 매출 기준 고객사 규모 등 8종이다. 개입 변수는 Tech Support와 Discount, 결과 변수는 이후 1년 동안의 소프트웨어 구매 매출 Revenue다.

무작위 실험(RCT)이 가장 이상적이지만, 실제 비즈니스 환경에서는 비용 문제, 영업 전략, 대형 고객사를 놓칠 위험 등으로 무작위 실험을 수행하기 어렵다. 따라서 이번 데이터는 무작위 실험 데이터가 아닌 관찰 데이터다.

Tech Support와 Discount는 모두 개입이므로, 이를 조합해 네 가지 처치로 정의한다($A_i \in \{0, 1, 2, 3\}$).

표 1. 네 가지 처치 정의

A_i	이름	의미
0	none	아무 개입도 제공하지 않음
1	tech_support_only	기술지원만 제공
2	discount_only	할인만 제공
3	discount_plus_support	기술지원과 할인을 모두 제공

2.1 비용 설정

수익 극대화를 위해서는 매출과 함께 비용도 고려해야 한다. 동일한 프로모션이라도 고객사 특성에 따라 비용이 달라질 수 있다. 이 데이터에는 실제 비용 정보가 없으므로, 기술지원 비용은 직원 수가 많을수록, 할인 비용은 고객사 규모가 클수록 증가하도록 시뮬레이션한다. 비용이 항상 양수이고 일부 고객사에서 큰 비용이 발생하는 분포를 반영하기 위해 로그정규분포를 사용한다.

$$C_i(\text{tech}) \sim \text{LogNormal}(\log C_{\text{tech}} + 0.5 \tilde{e}_i, 0.3), \quad C_i(\text{disc}) \sim \text{LogNormal}(\log C_{\text{disc}} + 0.4 \tilde{s}_i, 0.3)$$

여기서 $\tilde{e}_i$ 는 표준화된 직원 수, $\tilde{s}_i$ 는 표준화된 회사 규모다. 고객사별 비용을 반영한 최종 순이익(net outcome)은 $Y_i^{\text{net}} = Y_i - C_i(A_i)$ 로 정의한다. 이렇게 하면 이후 정책 학습과 평가가 순이익 기준으로 이루어져, 비용을 고려한 최적화가 가능해진다. 정책 학습과 평가는 train/test를 50:50으로 분리해 (각 1,000곳) 과대추정을 방지한다.

3. Q1. 프로모션 효과는 정말 고객사마다 다른가?

타겟팅이 의미가 있으려면 먼저 처치 효과가 고객사마다 다르다는 신호가 있어야 한다. 처치별 표본은 23~27%로 고르게 분포해(표 2) 비교에 안정적이다. 한편 처치별 평균 매출은 처치 없음일 때 6,586, 기술지원+할인일 때 26,784로, 처치에 따라 4배 이상 차이가 난다.

표 2. 처치별 표본 수·평균 매출·평균 비용·평균 순이익

처치	표본 수	평균 매출	평균 비용	평균 순이익
none	517	6,586	0	6,586
tech_support_only	462	15,104	5,038	10,066
discount_only	477	12,248	5,182	7,066
discount_plus_support	544	26,784	12,665	14,119

이 평균 차이를 그대로 처치 효과로 해석하면 안 된다. 그림 1에서 보듯 처치 그룹별 고객사 특성이 다르고(특히 Size, IT Spend), 규모가 큰 고객사일수록 더 강한 개입을 받는 경향이 있다. 즉 매출 차이에는 고객사 특성의 영향과 실제 처치 효과가 함께 섞여 있다(그림 2).

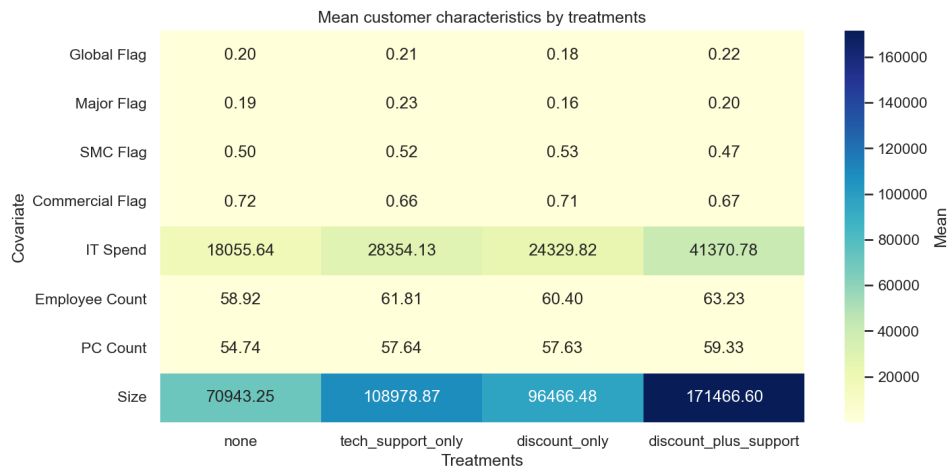


그림 1. 처치별 고객사 특성 평균. Size, IT Spend가 처치 그룹마다 다르게 나타나며, 처치 배정이 고객사 특성과 얽혀 있음을 보여준다.

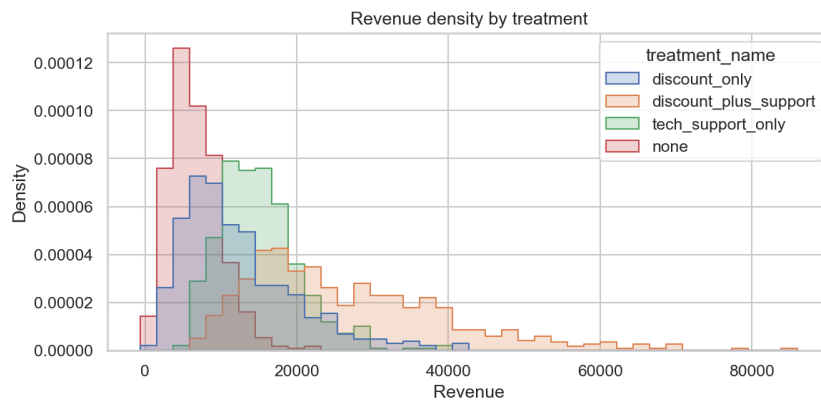


그림 2. 처치별 Revenue 분포. 규모·분산·이상치가 처치마다 다르다.

Q1 요약. 처치 그룹 간 고객사 특성과 결과 분포가 뚜렷이 다르다. 이는 (a) 처치 효과가 고객사마다 다를 가능성과 (b) 처치 배정이 고객사 특성에 의해 교란되어 있음을 동시에 시사한다. 따라서 단순 평균 비교가 아니라 교란을 보정하는 인과적 접근이 필요하다.

4. Q2. 관찰 데이터로 인과 효과를 신뢰할 수 있는가?

관찰 데이터에서 처치 효과를 인과적으로 해석하려면 두 가지 가정이 필요하다.

1. Unconfoundedness (비교란성). $\{Y_i(0), Y_i(1), Y_i(2), Y_i(3)\} \perp A_i \mid X_i$. 관측된 공변량 X_i 를 조건부로 했을 때 처치 배정이 잠재 결과와 독립이어야 한다. 즉, 고객사 규모 · IT 지출 · 직원 수 등이 처치 배정을 충분히 설명한다고 가정한다.

2. Positivity (양수성). $P(A_i = a \mid X_i = x) > 0, \forall a, \forall x$. 모든 고객사 특성 범위에서 각 처치가 어느 정도 관측되어야 한다. 이는 propensity score $e_a(x) = P(A_i = a \mid X_i = x)$ 를 추정해 직접 확인할 수 있다.

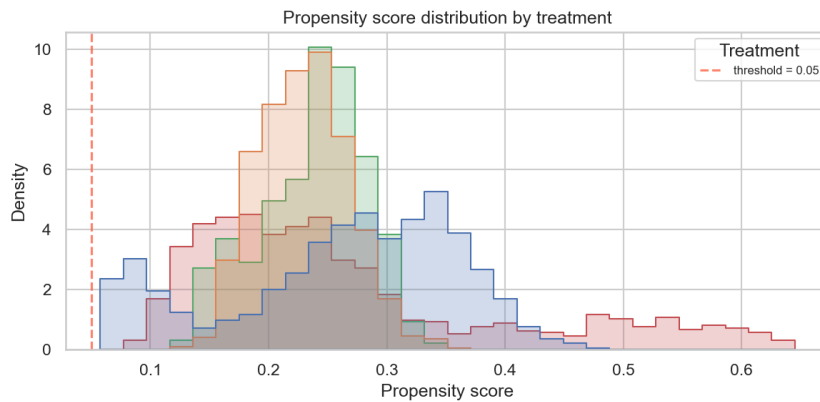


그림 3. 처치별 propensity score 분포. 네 처치의 분포가 잘 겹치며, $\hat{e}_a(X_i)$ 의 최솟값은 약 0.058로 0.05 미만 관측치가 없다.

이 데이터에서 $\hat{e}_a(X_i)$ 의 최솟값은 약 0.058이며, 0.05 미만 관측치도 없다(그림 3). 즉 극단적으로 작은 propensity score로 일부 관측치에 과도한 가중치가 부여되는 문제는 크지 않다.

Q2 요약. Positivity는 데이터로 확인되어 충족된다. Unconfoundedness는 검정 불가능한 가정이지만, 처치 배정을 설명할 핵심 변수들이 관측되어 있어 합리적인 가정으로 받아들인다. 따라서 이후 정책 학습과 평가는 outcome model과 propensity model을 함께 쓰는 DR(doubly robust) 방식으로 진행한다.

5. Q3. 누구에게 무엇을 배정해야 하는가?

세 가지 DR(doubly robust) 기반 정책을 학습해 비교한다. DR 방식은 outcome model과 propensity model을 함께 사용해 관찰 데이터의 처치 배정 편향을 보정한다. 이때 정책 학습의 기준이 되는 AIPW(Augmented Inverse Probability Weighting) score는 다음과 같다.

$$\hat{\Gamma}_{i,a} = \hat{\mu}_a(X_i) + \frac{\mathbf{1}[A_i = a]}{\hat{e}_a(X_i)} (Y_i^{net} - \hat{\mu}_a(X_i))$$

여기서 $\hat{\mu}_a(X_i)$ 는 고객사 i 가 처치 a 를 받았을 때의 예상 순이익, $\hat{e}_a(X_i)$ 는 고객사 i 가 실제로 처치 a 를 받을 확률(propensity score)이다. AIPW score는 고객사-처치 조합별 순이익을 DR 방식으로 보정한 점수이며, 점수가 높을수록 해당 처치의 기대 순이익이 높다고 해석한다.

세 가지 정책은 이 점수를 서로 다른 방식으로 사용한다.

- **Plug-in Policy (DRLearner):** 고객사별 CATE를 추정한 뒤 기대 순이익이 가장 큰 처치를 선택. 유연하지만 CATE 추정 품질에 민감.
- **DRPolicyTree:** AIPW score를 직접 최대화하되 정책을 얇은 의사결정 트리로 제한 $\Rightarrow$ 해석 가능.
- **DRPolicyForest:** AIPW score를 직접 최대화하되 forest 사용 $\Rightarrow$ 분산이 작고 안정적이지만 해석은 어려움.

5.1 Plug-in Policy

먼저 ‘처치 없음(none)’을 기준으로 삼아, 각 프로모션이 처치 없음 대비 순이익을 얼마나 더 늘리는지(상대효과)를 DRLearner로 추정한다. 즉 $\hat{\tau}_a(x) = \hat{\mathbb{E}}[Y^{net}(a) - Y^{net}(0) \mid X = x]$ 이다. 그런 다음 고객사마다 네 선택지의 기대 이득 $[0, \hat{\tau}_1(x), \hat{\tau}_2(x), \hat{\tau}_3(x)]$ 를 비교해 가장 큰 값을 주는 처치를 고른다. 첫 항이 0인 이유는 ‘처치 없음’이 기준이라 자기 자신과의 차이가 0이기 때문이다.

$$\hat{\pi}_{\text{plugin}}(x) = \arg \max_{a \in \{0,1,2,3\}} \hat{\mathbb{E}}[Y^{net}(a) - Y^{net}(0) \mid X = x]$$

이 정책은 약 50%에게 기술지원, 37%에게 기술지원+할인, 약 9%에게 처치 없음을 배정한다. 기대 순이익이 음수인 일부 고객사에게는 처치하지 않는 선택이 나타난다.

5.2 Policy Tree

현업에서는 성능뿐 아니라 설명 가능성도 중요하다. 얇은 의사결정 트리는 규칙이 단순해 이해관계자에게 설명하기 쉽고, 같은 기준을 반복 적용하기 쉽다.

DRPolicyTree는 고객사-처치 조합별 AIPW score를 만든 뒤, 제한된 깊이의 트리 규칙 중 평균 score가 가장 높은 규칙을 학습한다. 이 분석에서는 규칙이 복잡해지지 않도록 depth=2로 제한했다.

학습된 트리 구조는 그림 4와 같다. 트리는 주로 Size를 기준으로 분기하며, 규모가 큰 고객사에는 기술지원+할인을, 그 외 고객사에는 주로 기술지원을 배정한다.



그림 4. depth=2 DRPolicyTree. 주로 Size를 기준으로 분기한다. 규모가 큰 고객사에게는 기술지원+할인을, 그 외에는 주로 기술지원을, 기대 순이익이 낮은 일부(약 20%)에게는 처치 없음을 배정한다.

5.3 Policy Forest와 핵심 변수

Policy Forest는 단일 트리 대신 여러 트리를 평균해 분산이 작고 복잡한 이질성을 안정적으로 학습한다. 단일 규칙으로 해석하기는 어렵지만, 변수 중요도(feature importance)로 어떤 특성이 배정을 주도하는지 파악할 수 있다.

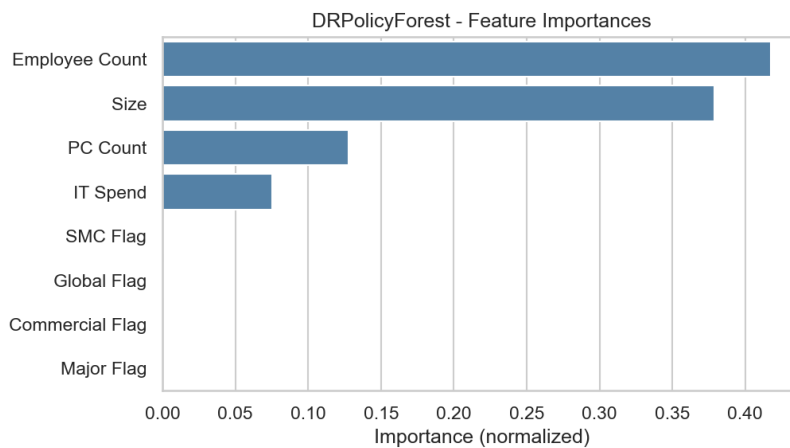


그림 5. DRPolicyForest 변수 중요도. Employee Count(42%)와 Size(38%)가 전체의 약 80%를 차지한다.

Employee Count(42%)와 Size(38%)가 전체 중요도의 약 80%를 차지하며, 고객사 규모와 직원 수가 처치 배정을 주도함을 확인할 수 있다(그림 5). 이는 트리 정책이 Size로 분기한 결과(그림 4)와도 일치한다.

Q3 요약. 세 정책 모두 “규모가 큰 고객사 → 기술지원+할인, 그 외 → 주로 기술지원, 기대 순이익이 낮은 일부 → 처치 없음”이라는 일관된 배정 패턴을 학습했다.

6. Q4. 학습한 정책이 정말 더 나은가?

학습된 정책은 동일한 test set에서 평가한다. 각 정책의 가치는 test 고객사별로 선택된 처치의 AIPW score를 평균해 계산한다. 이 값으로 학습 정책과 모든 고객사에게 동일 처치를 적용하는 baseline 정책을 비교한다.

95% 신뢰구간은 학습 과정을 포함한 bootstrap으로 계산한다. train set을 100번 재표본추출하고, 매번 정책과 평가용 nuisance model을 다시 학습한 뒤, 동일한 test set에서 정책 가치를 평가한다.

표 3. 정책별 AIPW 가치 비교와 학습 포함 bootstrap 95% 신뢰구간

정책	AIPW value	95% 신뢰구간
DRLearner plug-in	11,798	[11,464, 11,984]
DRPolicyForest	11,619	[11,364, 11,907]
DRPolicyTree (depth=2)	11,149	[11,056, 11,923]
all_tech_support_only	10,527	[10,316, 10,870]
all_discount_plus_support	10,137	[9,895, 10,388]
all_discount_only	7,545	[7,366, 7,778]
all_none	7,249	[7,133, 7,392]

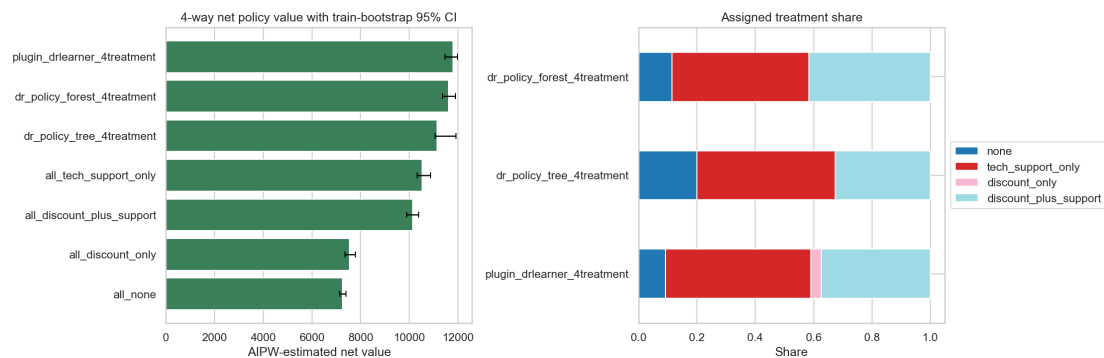


그림 6. (좌) 정책별 AIPW 순가치와 학습 포함 bootstrap 95% 신뢰구간. (우) 학습 정책의 처치 배정 비중.

세 학습 정책 모두 단일 처치 정책보다 높은 가치를 가진다(표 3, 그림 6). 고객사별 타겟팅이 모두에게 같은 처치를 적용하는 것보다 효과적이다. plug-in과 forest는 신뢰구간이 상당 부분 겹쳐 실질적 차이가 있다고 보기는 어렵다. 트리 정책은 해석 가능성을 유지하는 대신 성능을 일부 양보했다. all_discount_plus_support가 all_tech_support_only보다 낮은 이유는, 할인 비용이 고객사 규모에 따라 커지도록 설정해 일부 고객사에서 할인의 순수익이 비용을 상쇄하지 못했기 때문이다.

Q4 요약. 세 학습 정책 모두 최선의 단일 처치 정책보다 높은 순이익을 낸다(plug-in 기준 +12%, 처치 없음 대비 +63%). 타겟팅의 가치가 데이터로 확인된다.

7. Q5. 예산이 제한되면 어떤 정책이 유리한가?

앞의 Q4에서는 각 정책의 전체 평균 순이익을 비교했다. 그러나 실제 운영에서는 모든 고객사에게 정책을 그대로 적용할 만큼 예산이 충분하지 않을 수 있다. 이 경우 중요한 질문은 “어떤 정책의 평균 가치가 가장 높은가”가 아니라, “같은 예산 안에서 어떤 정책이 더 많은 순이익을 만드는가”이다.

이를 비교하기 위해 비용 곡선(cost curve)을 사용한다. 각 정책이 처치를 배정한 고객사에 대해, 앞에서 정의한 비용 함수를 바탕으로 두 값을 계산한다.

- **예상 비용:** 고객사 특성을 바탕으로 예측한 해당 처치의 비용
- **순이익 증가분:** 처치 없음 대비 AIPW 기준 순이익 증가분

그다음 고객사를 **순이익 증가분 / 예상 비용** 비율이 높은 순서로 정렬한다. 즉, 같은 비용으로 더 큰 순이익을 낼 수 있는 고객사부터 예산을 배정한다. 이 정렬 결과를 비용 곡선(그림 7)으로 시각화한다. 그림에서 x축은 누적 처치 비용, y축은 누적 순이익 증가분이며, 같은 x값에서 y값이 높을수록 같은 예산으로 더 높은 순이익을 내는 정책이다.

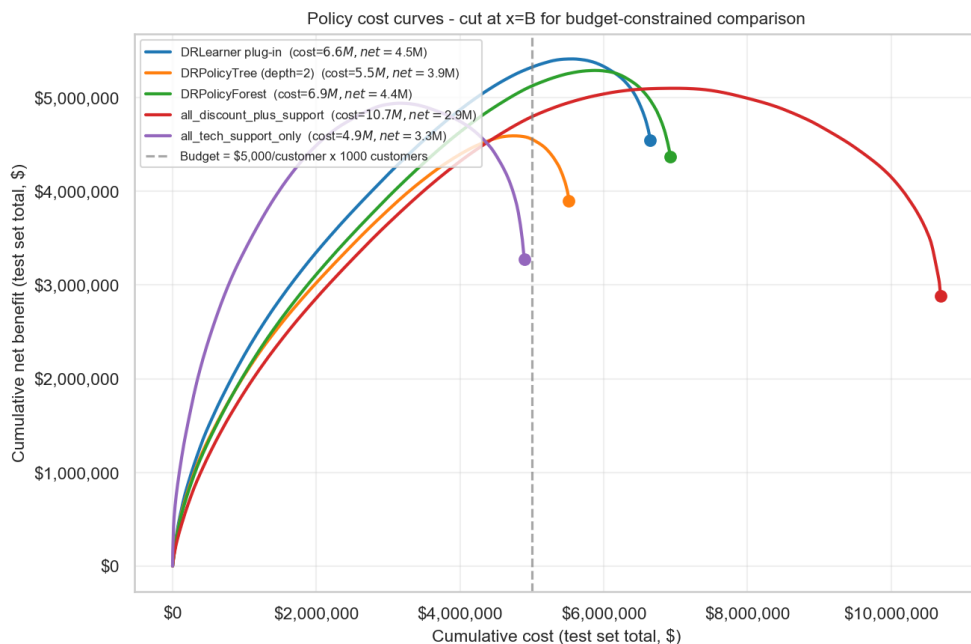


그림 7. 정책별 비용 곡선. 예산($x = B$)에서 y값이 높을수록 효율적이다. 학습 정책은 약 \$3.9M 이후 전원 기술지원을 앞서지만, 중상위 예산 구간에서는 기술지원+할인 정책이 일시적으로 가장 높다.

왜 순이익만으로 정렬하지 않는가? 예산이 제한되어 있을 때는 순이익의 절대 크기보다 비용 대비 효율이 중요하다. 예를 들어 A 고객사는 순이익 증가분이 \$10,000이지만 비용이 \$100,000이고, B 고객사는 순이익 증가분이 \$3,000이지만 비용이 \$5,000이라고 하자. 순이익만 보면 A가 더 좋아 보이지만, 예산이 \$10,000이라면 A는 처치할 수 없고 B는 처치할 수 있다. 따라서 예산 제약 하에서는 순이익이 큰 고객사이 아니라, 비용 대비 순이익이 큰 고객사부터 선택해야 한다.

예산 규모에 따라 효율적인 정책이 다르다.

- **소액 예산(\$3.9M 이하):** all_tech_support_only가 가장 효율적이다. 기술지원 평균 비용이 낮아 순수익/비용 비율이 높은 고객사부터 빠르게 처치할 수 있다.
- **중간 예산(\$3.9M ~ \$6.5M):** 학습 정책(plug-in, forest)이 앞선다. plug-in이 모든 고객사에 기술지원을 제공하는 정책을 추월하는 시점은 약 \$3.9M이다.
- **중상위 예산(\$6.5M ~ \$9.3M):** 기술지원+할인 정책이 일시적으로 가장 높다. 이 구간에서는 해당 정책 내에서 순이익/비용 비율이 높은 고객사들이 먼저 선택되기 때문이다.
- **예산 전부 소진:** 학습 정책이 가장 효율적이고, all_discount_plus_support는 효율이 가장 낮다(표 4). 기술지원+할인을 모든 고객사에 확대하면 비용이 커져 낮은 효율 고객까지 포함되기 때문이다.

표 4. 정책별 총 비용 · 총 순이익 (test set 1,000곳 누적 합계)

정책	총 비용	총 순이익
DRLearner plug-in	\$6.6M	\$4.6M
DRPolicyForest	\$6.9M	\$4.4M
DRPolicyTree (depth=2)	\$5.5M	\$3.9M
all_tech_support_only	\$4.9M	\$3.3M
all_discount_plus_support	\$10.7M	\$2.9M

all_discount_plus_support는 총 \$10.7M을 소진해도 순이익이 \$2.9M에 그쳐 효율이 가장 낮다. 할인 비용이 고객사 규모에 따라 커지도록 설정해 일부 고객사에서 순수익이 비용을 상쇄하지 못했기 때문이다.

Q5 요약. “어떤 정책이 최선인가”는 예산에 의존한다. 예산이 적으면 저비용 단일 처치가, 예산이 충분하면 타겟팅 정책이 유리하다. 비용 곡선은 예산을 고려한 의사결정에 직접 활용할 수 있는 도구다.

8. 결론 및 제언

약 2,000곳의 관찰 데이터에서, 비용을 반영한 순이익을 기준으로 네 가지 프로모션의 고객사별 최적 배정을 학습했다. 처치 효과의 이질성을 진단(Q1)하고 인과 식별 가정을 점검(Q2)한 뒤, 세 가지 정책을 학습(Q3)하고 학습/평가 데이터를 분리해 doubly robust(AIPW) 기준으로 가치를 비교(Q4)했으며, 예산 제약 하의 효율(Q5)까지 평가했다.

비즈니스 제언.

1. **고객사별 타겟팅이 단일 정책보다 효과적이다.** 최선의 단일 처치 대비 약 12%, 처치 없음 대비 약 63% 높은 순이익을 낸다.

2. **처치 배정의 핵심 신호는 고객사 규모와 직원 수다.** 규모가 큰 고객사에게는 기술지원+할인, 그 외에는 주로 기술지원이 적합하다. 기대 순이익이 낮은 일부는 처치 없음이 유리하다.
3. **예산 규모에 따라 최적 정책이 달라진다.** \$3.9M 이하라면 전원 기술지원이, \$3.9M~\$6.5M에서는 학습 정책이, \$6.5M~\$9.3M에서는 기술지원+할인 정책 내 고효율 고객사 선별이 유리하다.
4. **성능과 해석 가능성 사이에서 선택이 필요하다.** 성능 최우선이면 plug-in/forest가, 이해관계자 설명과 운영 안정성이 중요하면 depth=2 트리 정책이 적합하다.

한계와 다음 단계.

- 본 분석은 관찰 데이터 기반의 오프라인 추정이다. Unconfoundedness는 검정할 수 없는 가정이므로, 관측되지 않은 요인이 처치 배정과 성과에 동시에 영향을 미쳤다면 정책 가치가 편향될 수 있다.
- 비용은 실제 집행 비용이 아니라 시뮬레이션 값이다. 실제 할인 비용과 기술지원 운영 비용이 확보 되면 순이익과 예산 기준의 결론이 달라질 수 있다.
- 학습 포함 bootstrap 신뢰구간에서 상위 정책 간 구간이 일부 겹친다. 따라서 현재 결과는 정책 간 우열을 확정한다기보다, 타겟팅 정책이 단일 정책보다 유망하다는 근거로 해석하는 것이 적절하다.
- 실제 적용 이후에는 A/B 테스트가 가능하면 직접 검증하고, 어렵다면 AIPW 기반 사후평가와 성과 모니터링을 통해 정책을 주기적으로 점검해야 한다.

참고 자료

- Athey, S., & Wager, S. (2021). *Policy Learning with Observational Data*. Econometrica. arxiv.org/abs/1702.02896
- Sun, L., Du, X., Wager, S., et al. (2021). *Treatment Allocation under Uncertain Costs*. arxiv.org/abs/2103.11066
- Imai, K., & Li, M. L. (2019). *Experimental Evaluation of Individualized Treatment Rules*. arxiv.org/pdf/1905.05389
- Stanford ML+CI Tutorial — *Policy Learning I (Binary Treatment)*.
- Microsoft SynapseML. *Quickstart - Measure Causal Effects*. 공개 샘플 데이터: [multi-attribution sample CSV](#)

조해창 · 데이터 분석 포트폴리오 · 코드와 재현 가능한 분석: [GitHub Repository](#).